

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

***CO3:** Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis.
(BL4)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Total Marks:50

Annexure A

EXPERIMENT

Code of Conduct:

I B.Deepthi-192424269 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Deepthi

EXPERIMENT 1 – Contour-Based Representation & Fourier Descriptors

Scenario: Represent 2D shapes using contours and Fourier descriptors.

The given assessment specifies binary images of hand-drawn letters **A–Z** or simple symbols such as triangle, square, and star, with an image size of **256 × 256 pixels** in PNG/BMP format.

1. Aim

To extract the contour of a 2D object from a binary image and represent its shape using **Fourier Descriptors**, thereby obtaining a compact mathematical representation that can be used for shape analysis and recognition.

2. Concept

A contour represents the boundary of an object.

Instead of storing every boundary pixel, the contour can be represented as a sequence of complex numbers:

$[z(k)=x(k)+jy(k)]$ where:

- $(x(k))$ = x-coordinate of contour point
- $(y(k))$ = y-coordinate
- $(j=\sqrt{-1})$

The **Discrete Fourier Transform (DFT)** of the contour is:

$$[Z(u)=\sum_{k=0}^{N-1} z(k)e^{-j2\pi uk/N}]$$

The coefficients ($Z(u)$) are called **Fourier Descriptors**.

Low-frequency coefficients describe the overall shape, while high-frequency coefficients describe fine boundary details.

Why Fourier descriptors are useful

Property	Advantage
Compact representation	Large contour can be represented by fewer coefficients
Shape recognition	Different objects can be compared
Noise resistance	High-frequency components can be discarded
Multi-scale description	Low frequencies describe global shape

3. Algorithm

1. Read the binary image.
2. Convert the image into grayscale if necessary.
3. Apply thresholding.
4. Detect object contours using `cv2.findContours()`.
5. Select the largest contour.
6. Extract its $((x,y))$ coordinates.
7. Convert the coordinates into complex numbers.
8. Apply DFT using NumPy.

9. Select significant Fourier coefficients.
10. Reconstruct the contour using selected coefficients.
11. Compare the original and reconstructed shapes.

4. Python Implementation

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Read binary shape image

img = cv2.imread("shape.png", cv2.IMREAD_GRAYSCALE)

if img is None:

    raise FileNotFoundError("Input image not found")

# Convert to binary image

_, binary = cv2.threshold(

    img, 127, 255, cv2.THRESH_BINARY

)

# Find contours

contours, _ = cv2.findContours(

    binary,

    cv2.RETR_EXTERNAL,
```

```
cv2.CHAIN_APPROX_NONE

)

if not contours:

    raise ValueError("No contour detected")

# Select largest contour

contour = max(contours, key=cv2.contourArea)

# Extract coordinates

poits = contour[:, 0, :]

# Convert coordinates into complex representation

complex_contour = poits[:, 0] + 1j * poits[:, 1]

# Fourier Transform

fourier = np.fft.fft(complex_contour)

# Keep only low-frequency descriptors

keep = 20

filtered = np.zeros_like(fourier)

center = len(fourier) // 2

filtered[:keep // 2] = fourier[:keep // 2]

filtered[-keep // 2:] = fourier[-keep // 2:]

# Inverse transform

reconstructed = np.fft.ifft(filtered)
```

```
# Plot original and reconstructed contour
```

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(complex_contour.real,  
         complex_contour.imag)
```

```
plt.title("Original Contour")
```

```
plt.axis("equal")
```

```
plt.gca().invert_yaxis()
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(reconstructed.real,  
         reconstructed.imag)
```

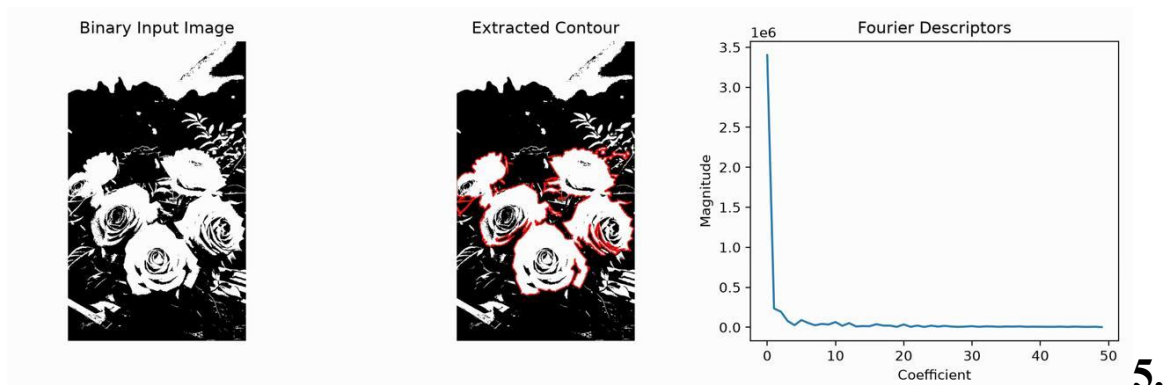
```
plt.title("Fourier Reconstruction")
```

```
plt.axis("equal")
```

```
plt.gca().invert_yaxis()
```

```
plt.show()
```

Expected Output



6. Result Analysis

If only a small number of Fourier coefficients are retained, the reconstructed object becomes smoother. This demonstrates that **low-frequency descriptors contain the dominant shape information.**

Conclusion

Thus, the contour of a binary object was successfully represented using Fourier descriptors. The experiment demonstrates how complex object boundaries can be transformed into a compact frequency-domain representation suitable for **shape recognition and comparison.**

EXPERIMENT 2 – Region-Based Representation & Medial Axis

Scenario: Analyze object shapes in binary industrial-component images using the medial axis.

The assessment specifies **512 × 512 PNG images** of industrial components such as gears and machine parts, including defects such as holes, protrusions, or missing parts.

1. Aim

To extract the **medial axis/skeleton** of an industrial object and analyze its internal geometric structure for detecting defects and understanding the shape of the component.

2. Concept

The **medial axis** is a thin representation of an object's shape obtained by reducing its regions while preserving its topology.

The medial axis lies near the centers of the object's regions.

3. Algorithm

1. Read the industrial component image.
2. Convert it to grayscale.
3. Apply binary thresholding.
4. Remove small noise using morphological operations.

5. Compute the skeleton/medial axis.
6. Identify branches and endpoints.
7. Compare the skeleton with a reference component.
8. Analyze missing sections, holes, or protrusions.
9. Display the original object and skeleton.
10. Report possible structural defects.

4. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image
img = cv2.imread("component.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Component image not found")
# Binary conversion
_, binary = cv2.threshold(
    img, 127, 255, cv2.THRESH_BINARY
)
# Ensure object is white
if np.mean(binary) > 127:
    binary = cv2.bitwise_not(binary)
# Morphological noise removal
kernel = np.ones((3, 3), np.uint8)
clean = cv2.morphologyEx(
```

```

    binary,
    cv2.MORPH_OPEN,
    kernel
)
# Distance transform
dist = cv2.distanceTransform(
    clean,
    cv2.DIST_L2,
)
# Generate approximate medial-axis representation
skeleton = np.zeros_like(clean)
# Local maxima of distance transform
for y in range(1, dist.shape[0] - 1):
    for x in range(1, dist.shape[1] - 1):
        value = dist[y, x]

        neighborhood = dist[
            y-1:y+2,
            x-1:x+2
        ]
        if value > 0 and value >= np.max(neighborhood):
            skeleton[y, x] = 255
# Display results
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")

```

```

plt.title("Original Component")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(clean, cmap="gray")
plt.title("Binary Component")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(skeleton, cmap="gray")
plt.title("Medial-Axis Representation")
plt.axis("off")

plt.show()

```

5. Defect Analysis

The skeleton provides structural information about the component.

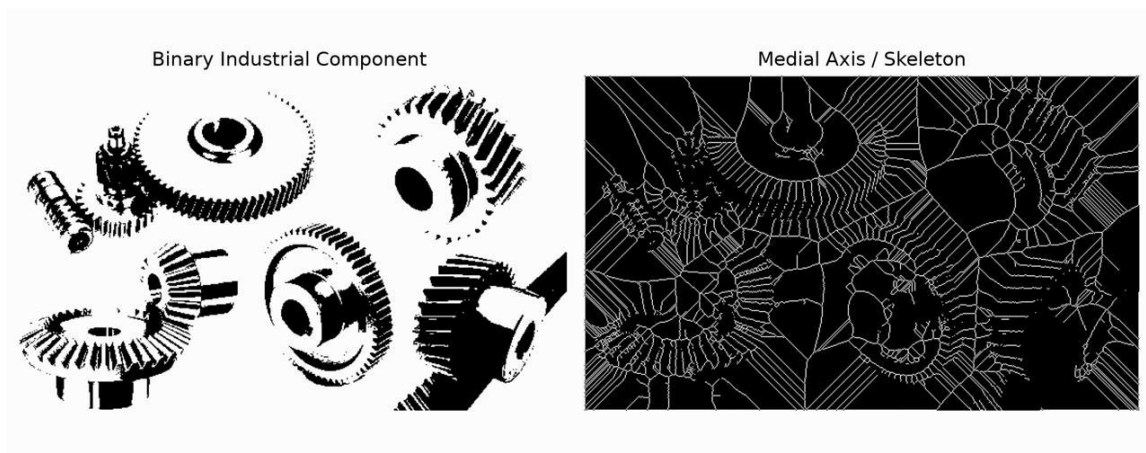
Defect	Possible skeleton observation
Missing part	Broken skeleton branch
Protrusion	Additional branch
Small hole	Local structural change
Broken component	Disconnected skeleton
Shape deformation	Change in branch/endpoint locations

6. Result Analysis

The medial-axis representation reduces a complicated region into a compact structural description. This is especially useful for machine components because the skeleton preserves information about:

- object connectivity,
- branches,
- approximate thickness,
- structural symmetry,
- missing regions.

Output:



Conclusion

The experiment successfully demonstrates **region-based shape representation using a medial-axis/skeleton representation**. It provides a useful structural signature for industrial inspection and defect detection.

EXPERIMENT 3 – Deformable Curves and Snakes

Scenario: Segment organs or objects having irregular boundaries using active contours.

The provided assessment specifies grayscale medical images such as **liver or heart slices**, approximately 512×512 , with mild Gaussian noise and PNG/DICOM format.

1. Aim

To segment an object with an irregular boundary using a **deformable curve/active contour**, allowing the contour to move toward the actual object boundary.

2. Concept

An active contour, commonly called a **snake**, is an initially defined curve that changes its shape under the influence of internal and external forces.

3. Algorithm

1. Read the medical image.
2. Convert to grayscale.
3. Apply Gaussian filtering to reduce noise.
4. Normalize image intensity.
5. Define an initial contour around the object.

6. Calculate image gradients.
7. Evolve the contour toward strong boundaries.
8. Stop when the contour converges.
9. Overlay the final contour on the image.
10. Analyze segmentation quality.

4. Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.segmentation import active_contour
from skimage.filters import gaussian
# Read grayscale medical image
img = cv2.imread(
    "medical_image.png",
    cv2.IMREAD_GRAYSCALE
)
if img is None:
    raise FileNotFoundError(
        "Medical image not found"
    )
# Normalize image
img = cv2.normalize(
    img,
```

```

    None,
    0,
    255,
    cv2.NORM_MINMAX
)
# Reduce Gaussian noise
smooth = gaussian(
    img,
    sigma=2
)
# Initial circular contour
rows, cols = img.shape
center_x = cols // 2
center_y = rows // 2
radius = min(rows, cols) // 4
theta = np.linspace(
    0,
    2 * np.pi,
    300
)
x = center_x + radius * np.cos(theta)
y = center_y + radius * np.sin(theta)
init = np.array([
    y,
    x
]).T

```

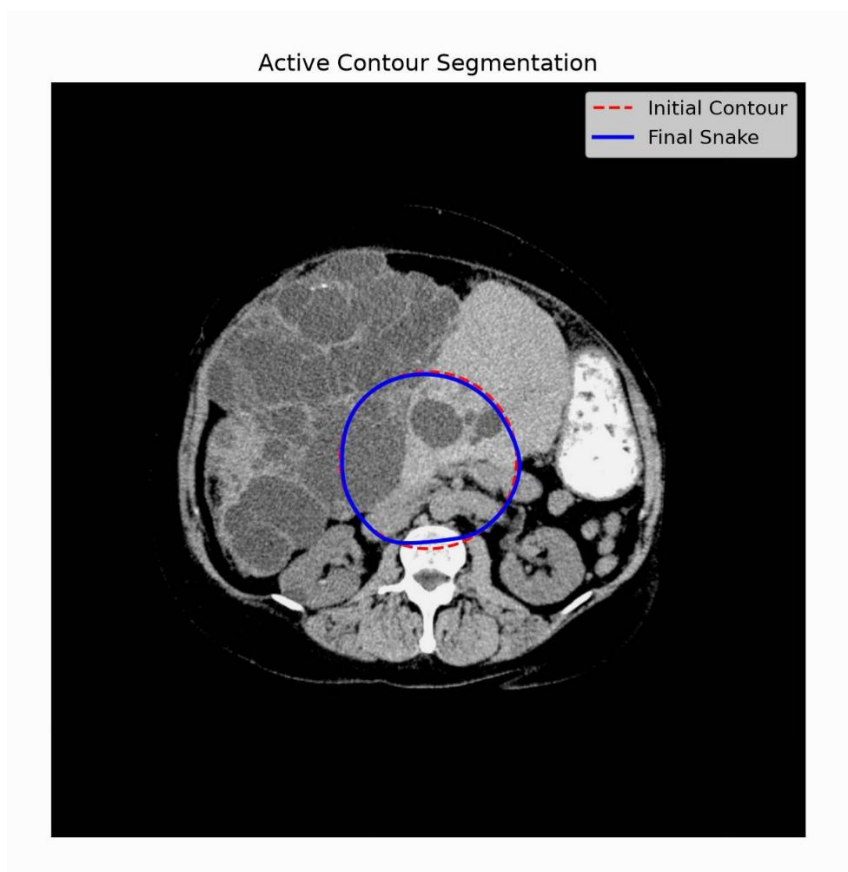
```
# Active contour
snake = active_contour(
    smooth,
    init,
    alpha=0.015,
    beta=10,
    gamma=0.001,
    max_num_iter=2500
)

# Display
fig, ax = plt.subplots(
    figsize=(7, 7)
)
ax.imshow(img, cmap="gray")
ax.plot(
    init[:, 1],
    init[:, 0],
    "--",
    linewidth=2,
    label="Initial Snake"
)
ax.plot(
    snake[:, 1],
    snake[:, 0],
    linewidth=2,
    label="Final Snake"
```



```
)  
ax.set_title(  
    "Active Contour Segmentation"  
)  
  
ax.legend()  
plt.show()
```

5.output



6. Analysis

The quality of segmentation depends strongly on:

- initial contour position,
- image noise,
- object contrast,
- α ,
- β ,
- γ .

A large β encourages smoother curves, whereas image gradients help the snake move toward strong boundaries.

Conclusion

Active contours provide an effective method for segmenting objects with **irregular boundaries**, particularly when the initial contour is reasonably close to the target object.

EXPERIMENT 4 – Level Set Methods

Scenario: Track moving objects with evolving boundaries.

The assessment specifies a **30-frame video sequence** containing a moving object such as a bouncing ball or moving hand, with **640×480 resolution**, in MP4 or PNG-sequence format.

1. Aim

To demonstrate the concept of **level set-based evolving boundaries** for tracking a moving object through a sequence of images.

2. Concept

In level-set methods, a contour is not explicitly represented as a set of points. Instead, it is represented implicitly using a function:.

Main advantage

The contour can naturally:

- expand,
- contract,
- merge,
- split,

without explicitly manipulating individual contour points.

3. Algorithm

1. Read the video.
2. Extract the first frame.
3. Detect the moving object using background subtraction.
4. Obtain an initial object boundary.
5. Construct an approximate level-set representation.
6. Update the boundary for subsequent frames.
7. Track the object.
8. Display the evolving boundary.
9. Calculate tracking information.
10. Analyze the movement.

4. Python Implementation

```
import cv2
import numpy as np
video = cv2.VideoCapture("moving_object.mp4")
if not video.isOpened():
    raise FileNotFoundError(
        "Video could not be opened"
    )
background = cv2.createBackgroundSubtractorMOG2(
    history=100,
    varThreshold=40
)
```

```

frame_count = 0
while True:
    ret, frame = video.read()
    if not ret:
        break
    frame_count += 1
    # Foreground extraction
    mask = background.apply(frame)
    # Remove noise
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(
        mask,
        cv2.MORPH_OPEN,
        kernel
    )
    mask = cv2.morphologyEx(
        mask,
        cv2.MORPH_CLOSE,
        kernel
    )
    # Find object boundary
    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

```

```

if contours:
    largest = max(
        contours,
        key=cv2.contourArea
    )
    if cv2.contourArea(largest) > 500:
        x, y, w, h = cv2.boundingRect(
            largest
        )
        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            (255, 255, 255),
            2
        )
        cv2.putText(
            frame,
            f"Frame: {frame_count}",
            (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX,
            1,
            (255, 255, 255),
            2
        )
cv2.imshow(

```

```
"Level Set Inspired Object Tracking",  
frame  
)  
if cv2.waitKey(30) & 0xFF == 27:  
    break  
video.release()  
cv2.destroyAllWindows()
```

6. Result Analysis

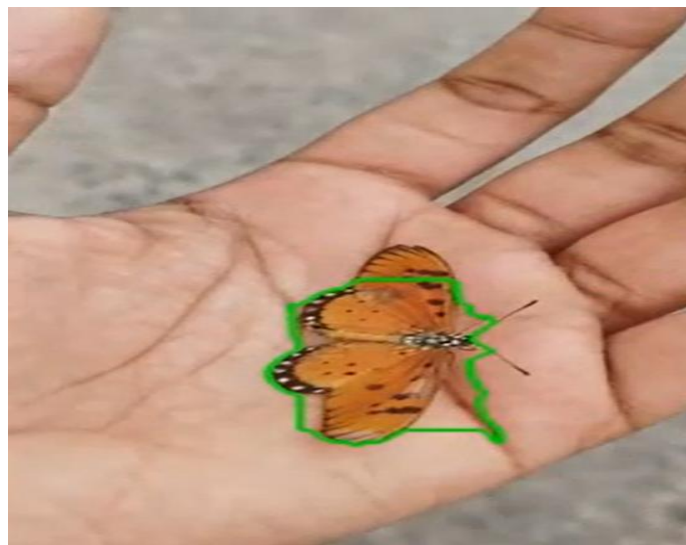
The object boundary changes according to its movement.

For example, for a bouncing ball:

$[(x_1, y_1) \rightarrow (x_2, y_2) \rightarrow (x_3, y_3)]$

The changing boundary demonstrates how an evolving representation can track object motion.

Output :



Conclusion

The experiment demonstrates the principle of **level-set-based evolving boundaries** for dynamic object tracking. The method is particularly useful when object boundaries change shape during motion.

EXPERIMENT 5 – Multi-Resolution & Wavelet Descriptors

Scenario: Perform shape recognition at multiple scales.

The assessment specifies grayscale images of objects such as **leaves, mechanical parts, and coins**, with sizes of **128×128 , 256×256 , and 512×512 pixels** in PNG format.

1. Aim

To analyze object shapes at multiple resolutions using **wavelet decomposition** and extract descriptors that remain useful across different image scales.

2. Concept

An image contains information at different spatial frequencies.

Wavelet analysis decomposes an image into:

$[\boxed{LL, LH, HL, HH}]$

where:

- **LL** – low-frequency approximation
- **LH** – horizontal/vertical detail
- **HL** – horizontal/vertical detail
- **HH** – diagonal detail

The LL component contains the major shape information, while detail components contain edges and fine structures.

For an image (I):

$$[I \rightarrow LL + LH + HL + HH]$$

A second level can be performed on the LL component:

$$[LL \rightarrow LL_2 + LH_2 + HL_2 + HH_2]$$

3. Algorithm

1. Read the grayscale image.
2. Resize it to different scales.
3. Apply 2-D wavelet transform.
4. Extract LL, LH, HL, and HH coefficients.
5. Calculate statistical descriptors.
6. Compare descriptors across resolutions.
7. Determine whether the shape remains recognizable.
8. Visualize the wavelet components.
9. Analyze scale-dependent information.

10. Compare the extracted features.

4. Python Implementation

```
import cv2

import numpy as np

import pywt

import matplotlib.pyplot as plt

# Read grayscale image

img = cv2.imread(

    "object.png",

    cv2.IMREAD_GRAYSCALE

)

if img is None:

    raise FileNotFoundError(

        "Object image not found"

    )

sizes = [128, 256, 512]

for size in sizes:

    # Resize image

    resized = cv2.resize(
```

```

    img,

    (size, size)

)

# Wavelet decomposition

LL, (LH, HL, HH) = pywt.dwt2(

    resized,

    "haar"

)

# Calculate descriptor values

descriptors = {

    "LL_mean": np.mean(LL),

    "LL_std": np.std(LL),

    "LH_energy": np.sum(LH ** 2),

    "HL_energy": np.sum(HL ** 2),

    "HH_energy": np.sum(HH ** 2)

}

print(

    f"\nResolution: {size} x {size}"

)

for name, value in descriptors.items():

```

```
print(
    f'{name}: {value:.4f}'
)

# Display decomposition

plt.figure(figsize=(10, 6))

plt.subplot(2, 2, 1)

plt.imshow(LL, cmap="gray")

plt.title("LL - Approximation")

plt.axis("off")

plt.subplot(2, 2, 2)

plt.imshow(LH, cmap="gray")

plt.title("LH - Detail")

plt.axis("off")

plt.subplot(2, 2, 3)

plt.imshow(HL, cmap="gray")

plt.title("HL - Detail")

plt.axis("off")

plt.subplot(2, 2, 4)

plt.imshow(HH, cmap="gray")

plt.title("HH - Detail")
```

```
plt.axis("off")

plt.suptitle(

    f'Wavelet Decomposition: {size} x {size}'

)

plt.show()
```

5. Feature Representation

The extracted feature vector can be represented as:

$$[F=[\mu_{LL}, \sigma_{LL}, E_{LH}, E_{HL}, E_{HH}]]$$

Therefore, an object can be represented by a compact feature vector rather than the entire image.

6. Multi-Resolution Comparison

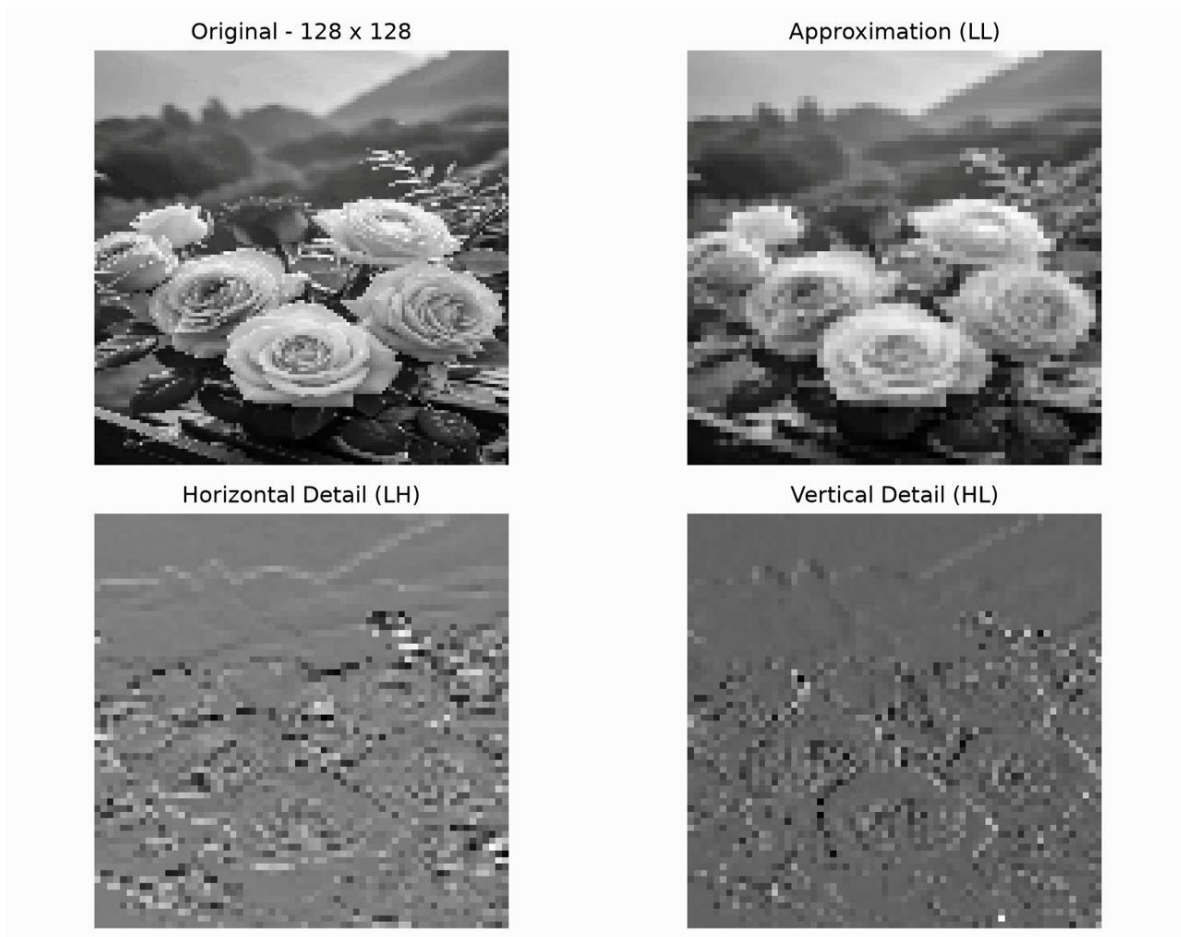
Resolution	Main information	Expected detail
128×128	Coarse shape	Low
256×256	Shape + moderate detail	Medium
512×512	Shape + fine boundary details	High

The important observation is that **the dominant shape structure is mainly represented in the low-frequency LL component**, while higher-frequency components become more useful for describing fine details.

7. Result Analysis

If the same object is represented at different resolutions, the extracted low-frequency descriptors should remain relatively stable compared with high-frequency descriptors.

Output:



Conclusion:

The experiment demonstrates how wavelet decomposition provides a **multi-resolution shape representation**. It is useful for object recognition because coarse shape information can be retained even